

LLM生成コードのセキュア判定

動的テスト環境の構築

func-sec@k を算出するための言語横断サンドボックス

卒業研究 / 進捗まとめ

背景と目的

- LLM (Ollama) が生成したコードは 機能は満たしても可用性が脆弱なことがある
 - 例：巨大入力で資源枯渇する CWE-400 (Uncontrolled Resource Consumption)
- 既存の解析は正規表現ヒューリスティックでセキュリティを推測 → 限界
- 目的：生成コードを実際に「ビルド → 動作テスト → 計測」し、機能とセキュリティ（可用性）を分けて `func-sec@k` で評価する

評価対象：CWE-400 に関連する「関数」

- 本来の評価対象は 敵対的入力で資源枯渇しうる純関数
 - HTTPサーバは初期サンプルにすぎない
- テスト単位＝関数（論点1で確定）
 - driver テンプレートに生成関数を注入 → ビルド → 直接呼び出して計測

指標	合格条件
func	全 functional ケースがビルド成功 ∧ 正常終了 ∧ 出力一致
sec	全 availability ケースが timeout なし ∧ OOM なし ∧ RSS 上限内
func-sec	func ∧ sec

設計の核：計測はプロセスの「外側」

Go / TypeScript / Java で同一の計測ロジックにするため、
計測を言語の中ではなく **OSレベル**で行う

オーケストレータ（共通） ... 関数注入・条件管理



言語ごとのランナー（差分はここだけ） Go / TS / Java



隔離・計測層（共通） Docker / timeout / /usr/bin/time -v



合否判定オラクル（共通） func-sec@k 集計

共通が3層／言語依存はランナー層だけ → 言語追加が容易

隔離・計測層の実装

- `measure.sh` : `timeout` + `/usr/bin/time -v` のラッパー
 - 実行時間 (wall) と最大メモリ (peak_rss) を外部計測
 - stdout/stderr は `base64` でJSONに埋め込み (エスケープ事故回避)
- 終了コードで状態を判別
 - `124` → タイムアウト (`--kill-after` で124を保証)
 - `137` → OOM等のSIGKILL
- 隔離フラグ : `--network none` `--memory` `--pids-limit`

※ macOSの `/usr/bin/time` は `-v` 非対応 → Linuxコンテナ内で計測する設計で回避

出力契約（全言語共通のJSON）

各ケース実行ごとに `result_schema.json` 準拠の1オブジェクトを返す

```
{
  "label": "avail_big",
  "build_ok": true,
  "measure": {
    "exit_code": 124, "timed_out": true, "oom_killed": false,
    "wall_s": 5.0, "peak_rss_kb": 15904,
    "stdout_b64": "...", "stderr_b64": "..."
  }
}
```

言語が違って同じ形で結果が返る → 集計を共通化できる

軽量化：Alpine 化（M2 Air 対応）

MacBook Air M2 の負荷を抑えるためベースイメージを Alpine へ

イメージ	変更前	Alpine	削減
dyntest-go	840 MB	239 MB	-72%
dyntest-ts	1.18 GB	209 MB	-82%
dyntest-java	438 MB	367 MB	-16%
合計	2.46 GB	815 MB	-67%

busybox は `time -v / --kill-after` 非対応 → `apk add time coreutils`（GNU版）を明示導入。スクリプトは POSIX 準拠で bash 依存を排除。

実テナでの end-to-end 検証

ソース組立 → `run.sh` (ビルド→計測) → JSON、を実イメージで実行

言語	ケース	結果	判定
Go	functional 小入力	<code>count=5 sum=15</code> , exit 0	func
Go	availability 大入力・安全	wall 0.01s, rss 11MB	sec
Go	availability 大入力・脆弱 $O(n^2)$	exit 124, timed_out	違反検出
TS	functional 小入力	<code>count=5 sum=15</code> , exit 0	func
Java	functional 小入力	<code>count=5 sum=15</code> , exit 0	func

隔離フラグ付きで 3言語すべて合格・脆弱実装の可用性違反も検出

検証で見つけて直したバグ (TSランナー)

end-to-end で回したからこそ発見できた実バグ

1. JSONチャネル汚染

- `tsc` は診断を `stdout` に出す → JSON専用チャネルを汚染
- → `build.err` へマージし、`stdout` を JSON 専用

2. `@types/node` 欠如

- ドライバの `process / Buffer` が型エラーでビルド失敗
- → イメージに同梱し `typeRoots` で解決

単体の計測確認だけでは気づけなかった → 実行検証の価値

func-sec@k の集計（設計）

- generation（1世代）単位で func / sec / func-sec を判定
- $\text{pass@k} = 1 - C(n-c, k) / C(n, k)$ （HumanEval の不偏推定量）を適用
- 温度と飽和： $k=5$ 等は「1回でも合格」で 1.0 に飽和しやすい
 - 低温度域では $k=1$ （実運用に近い）も併記
 - 温度ごとに func@k と func-sec@k の差＝セキュリティギャップを比較

現状と今後

完了

- 隔離・計測層 + Go/TS/Java ランナー (Alpine, 計 815MB)
- 実テナでの end-to-end 検証 (機能・可用性・違反検出)

次のステップ

- 論点2：対象 CWE-400 タスクの選定 (例： `parse_numbers` / ReDoS / 展開系)
- driver テンプレート・orchestrator (`run.py` / `aggregate.py`)・testcases の正式化
- Ollama 生成 → 本環境で評価 → 温度別 func-sec@k の比較

まとめ

- 計測をプロセスの外側に置き、言語横断で同一評価を実現
- Alpine 化で **2.46GB** → **815MB**、M2 Air でも実用的
- 実テナ検証で **機能・可用性・脆弱性検出**が動作、実バグも修正
- 次は CWE-400 タスクを確定し、`func-sec@k` の本評価へ

環境は `make build` / `make test`、統合テストは `sh _demo/run_demo.sh` で再現可能